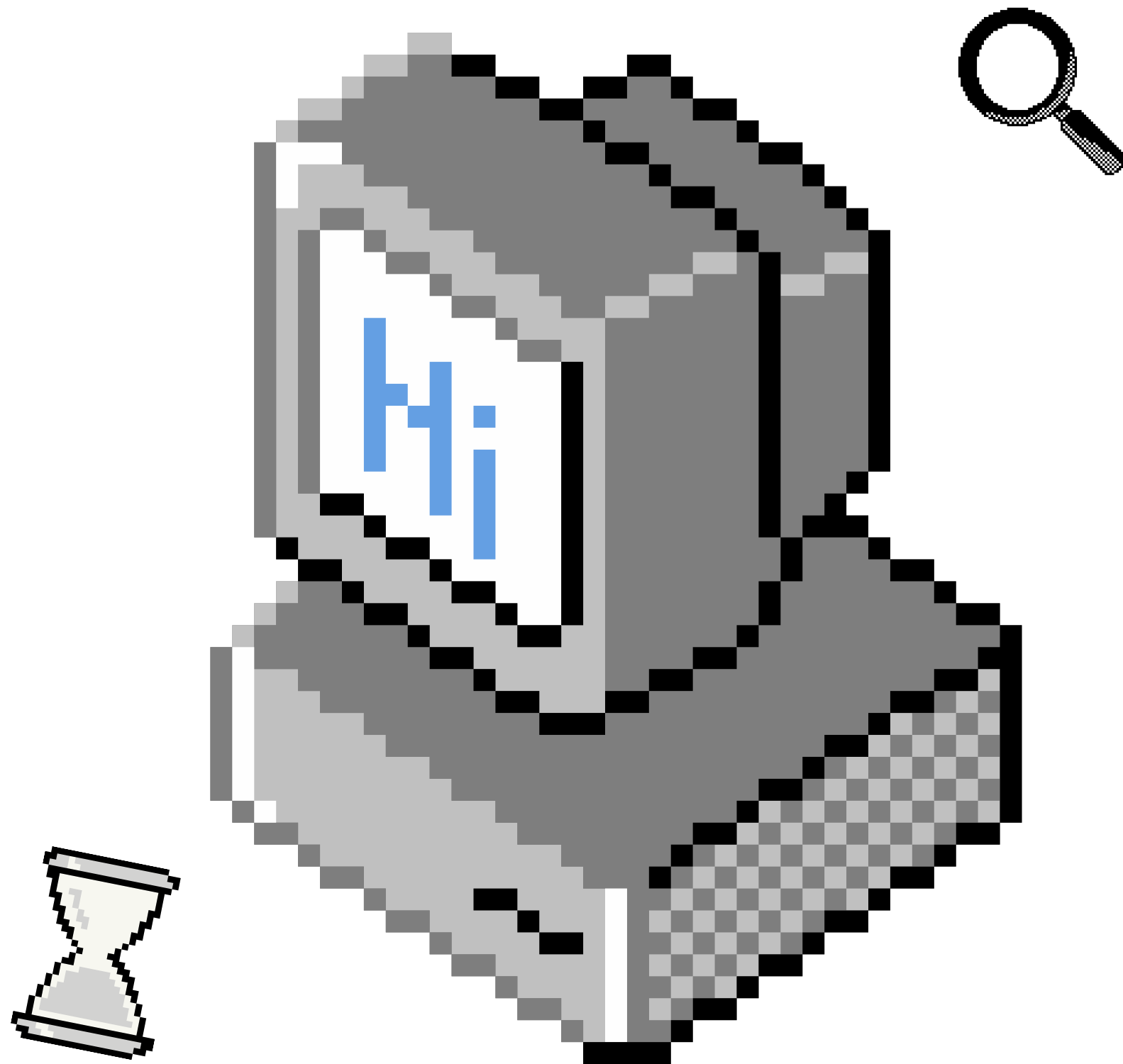




DEAP

(DISTRIBUTED EVOLUTIONARY ALGORITHMS IN PYTHON)

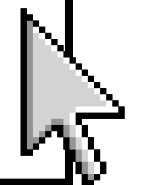
An open-source Python library designed to easily implement genetic algorithms and evolutionary computation.

Arshan Mukadam, Esther Cho

1. INSTALLATION

```
# Create and activate a virtual environment
python -m venv deap-env
source deap-env/bin/activate          # macOS/Linux
# deap-env\Scripts\activate           # Windows

# Install DEAP and verify
pip install deap
python -c "import deap; print(deap.__version__)" # should print e.g. 1.4.1
```

1. Create and activate a virtual env
 2. Install DEAP and verify
- 

2. PROBLEM&ALGORITHM OVERVIEW

Problem: OneMax

- Individual = bitstring of length 50
- Goal: evolve population toward the all-1s string
- Fitness = number of 1-bits → maximization problem
- Classic toy benchmark for demonstrating GA loops

What DEAP works here?

- Modular Setup: Supplies generic EA building blocks; user plugs in representation, evaluation, and operators.
- Execution(eaSimple): Runs standard GA loop (Select→Crossover→Mutate→Evaluate and Replace)
- Tracking: Evolves population across 50 generations while monitoring avg/max fitness.



3. CODE STRUCTURE OVERVIEW

- The first four sections (Setup, Toolbox registrations, GA operators, and Custom Function) all just register things before `main()` runs
- The actual evolutionary loop starts with the `eaSimple()` call.

Setup

- `creator.create()` — defines `FitnessMax` and `Individual` classes
- `base.Toolbox()` — creates the registry

Toolbox registrations

- `random.randint` — generates each random gene (0 or 1)
- `tools.initRepeat` — builds individuals/populations by repetition

GA operators

- `tools.selTournament` — selection
- `tools.cxTwoPoint` — crossover
- `tools.mutFlipBit` — mutation

Custom function

- `evaluate()` — fitness function (`sum(individual)`)

Stats & run

- `tools.Statistics` — tracks avg/max fitness per generation
- `algorithms.eaSimple` — runs the full evolutionary loop

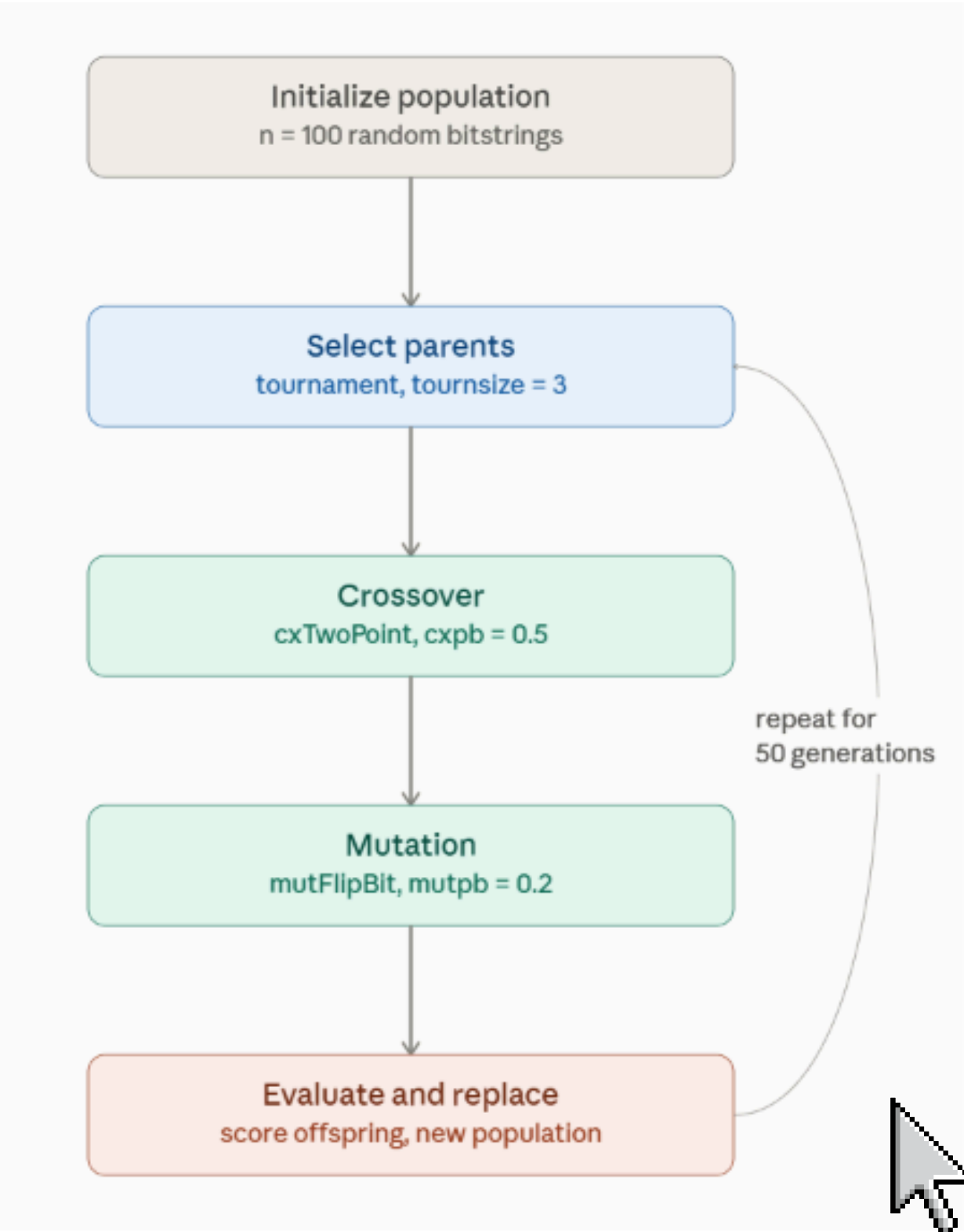
4. THE EVOLUTIONARY LOOP (EASIMPLE)

eaSimple() takes everything registered in the toolbox and runs the actual generational loop, repeating for *ngen* generations.

[Key Variables]

Variable	Meaning
<code>pop</code> (n=100)	Population size
<code>cxpb = 0.5</code>	Probability two selected parents crossover
<code>mutpb = 0.2</code>	Probability an offspring undergoes mutation
<code>ngen = 50</code>	Number of generations to run

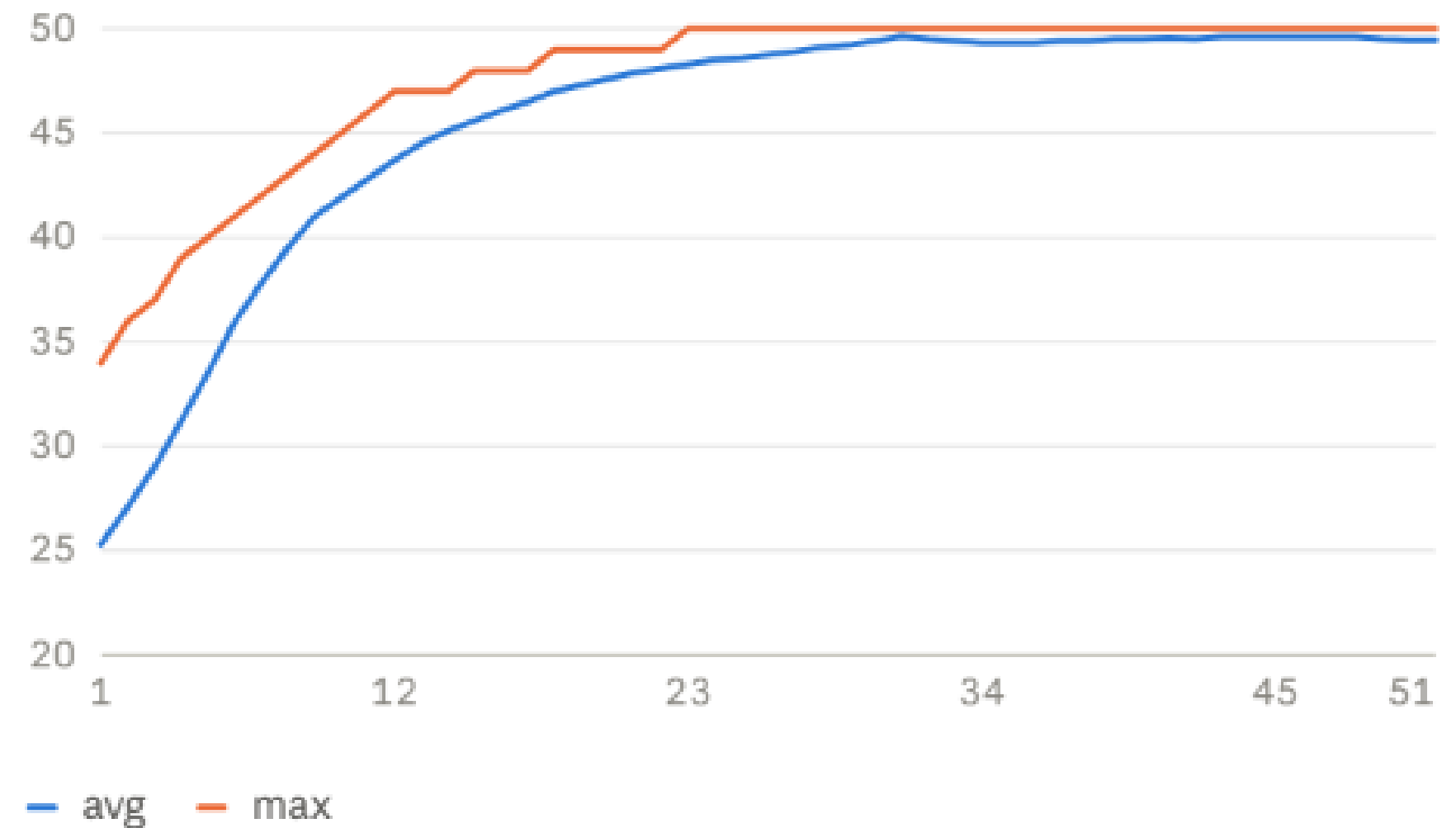
[Process Diagrams]



5. EXPECTED RESULTS

- *avg* and *max* fitness rise steadily each generation
- *max* reaches 50 (all-1s solution) around generation ~22, well before the 50-generation limit
- *avg* continues climbing more slowly as the population converges toward all-1s
- Demonstrates the core GA principle: selection pressure + crossover + mutation reliably drive a random population toward the optimum

[Convergence Chart]



DEMO

